

OSG/OSPool Quickstart Guide

A100 GPU Job Submission for Fermionic Adaptive Circuit Simulations

Asadullah Bhuiyan
ab2398@cornell.edu

April 2026

Contents

1	Account Setup and Login	2
2	HTCondor Basics	2
2.1	Minimal Submit File Structure	2
2.2	Job Duration Categories	3
2.3	Monitoring Commands	3
3	Targeting A100 GPUs	3
3.1	GPU Availability vs. Specificity Trade-off	3
4	Container Setup for Your Environment	4
4.1	Pre-Built Containers (Reference)	4
4.2	Building a Custom Container (Local Machine)	4
5	Submitting 100 A100 Jobs	5
5.1	Submit File: <code>batch_gpu.submit</code>	5
5.2	Executable: <code>run.sh</code>	5
5.3	Important Batch Submission Notes	6
6	Mapping Your Workflows to OSPool Jobs	6
6.1	Pure-State Markov Snapshot Runs (<code>colab_small_system_testing</code>)	6
6.2	Large $N=20$ Entanglement Scaling (<code>colab_large_entanglement_scaling_N20</code>) . . .	7
6.3	Strip Entanglement Post-Processing	7
7	Data Transfer	7
7.1	Small Files (<100 MB): <code>transfer_input_files</code>	7
7.2	Larger Files: OSDF	7
7.3	Transferring Results Back	8
8	Common Pitfalls and Fixes	8
9	Full Working Example	8
10	Quick Reference Card	9

1 Account Setup and Login

After your OSG account is approved (within one business day of request), you will be assigned to one of the UW access points. SSH in with:

```
ssh asad.bhuiyan@ap20.uw.osg-htc.org
# alternates: ap21.uw.osg-htc.org or ap23.uw.osg-htc.org
```

If SSH fails, upload your public key via the OSG portal web interface first, then retry. To generate a key if you do not have one:

```
ssh-keygen -t ed25519 -C "ab2398@cornell.edu"
# public key lives at ~/.ssh/id_ed25519.pub -- paste into portal
```

Check which project you belong to (needed for resource accounting):

```
grep $USER /etc/condor/UserToProjectMap.txt
```

2 HTCondor Basics

OSPool uses HTCondor for job scheduling. The core workflow is:

1. Write an **executable** script (`run.sh` or `run.py`).
2. Write a **submit file** (`*.submit`) declaring resources and pointing to the executable.
3. Run `condor_submit job.submit`.
4. Monitor with `condor_q` or `condor_watch_q`.

2.1 Minimal Submit File Structure

```
universe = container
container_image = /cvmfs/singularity.opensciencegrid.org/...

executable = run.sh
arguments = $(Process)

log = logs/job_$(Cluster)_$(Process).log
error = logs/job_$(Cluster)_$(Process).err
output = logs/job_$(Cluster)_$(Process).out

transfer_input_files = run.sh

+JobDurationCategory = "Medium"

request_gpus = 1
request_cpus = 4
request_memory = 16 GB
request_disk = 10 GB

queue 1
```

Important: `+JobDurationCategory` is *required* on all OSPool access points. Missing it causes the job to be held immediately.

2.2 Job Duration Categories

Category	Wall time
"Short"	< 1 hour
"Medium"	1–10 hours
"Long"	10–20 hours

Jobs exceeding 20 hours require HTCondor checkpointing or they will be evicted.

2.3 Monitoring Commands

```
condor_q                                # queue summary
condor_watch_q                          # live update (Ctrl+C to exit)
condor_q -hold                          # see why held jobs are held
condor_q -format "%d\n" JobStatus | sort | uniq -c
# status codes: 1=Idle, 2=Running, 4=Complete, 5=Held

condor_rm <cluster-id>                  # remove entire cluster
condor_rm <cluster-id>.<process>         # remove single job
condor_rm $USER                         # remove all your jobs
```

3 Targeting A100 GPUs

A100 GPUs have CUDA Compute Capability 8.0. Use these HTCondor ClassAd attributes to target them:

```
# Pin exactly to A100 (CC = 8.0):
gpus_minimum_capability = 8.0
gpus_maximum_capability = 8.0

# Require at least 40 GB GPU RAM (A100-40GB variant):
gpus_minimum_memory = 40000

# For the 80 GB variant:
gpus_minimum_memory = 80000
```

3.1 GPU Availability vs. Specificity Trade-off

Constraint	Hits	Notes
<code>capability = 8.0..8.0</code>	A100 only	Most restrictive, may queue longer
<code>capability ≥ 8.0</code>	A100 + A10 (22 GB)	Good throughput
<code>capability ≥ 7.0</code>	+ V100 (16 GB)	Widest pool

For `classA_U1FGTN_gpu` runs with `complex128` covariance matrices at $N_x=20$, $N_y \leq 120$, the A100-40GB is recommended. The V100 (16 GB) may be insufficient for large N_y .

4 Container Setup for Your Environment

Your code requires packages not present in any pre-built OSPool container (`pfapack`, `polars`, `h5py`, `torch 2.8`, `scipy`, etc.). You must build a custom container.

4.1 Pre-Built Containers (Reference)

CVMFS Path	Contents
<code>.../htc/pytorch:2.3.1-cuda11.8</code>	PyTorch 2.3.1, CUDA 11.8
<code>.../htc/tensorflow:2.15</code>	TensorFlow 2.15
<code>.../htc/rocky:9-cuda-12.6.0</code>	Rocky 9 + CUDA 12.6 (bare)
<code>.../htc/rocky:8-cuda-11.0.3</code>	Rocky 8 + CUDA 11.0 (bare)

Full CVMFS prefix: `/cvmfs/singularity.opensciencegrid.org/`

4.2 Building a Custom Container (Local Machine)

Docker is *not* available on OSPool access points. Build locally, push to Docker Hub, then pull on the access point.

Step 1 — Dockerfile (save as `Dockerfile`):

```
FROM hub.opensciencegrid.org/htc/rocky:9-cuda-12.6.0

# system deps
RUN dnf install -y python3 python3-pip && dnf clean all

# install your exact pinned environment
RUN pip install --no-cache-dir \
    torch==2.8.0 \
    numpy \
    scipy \
    matplotlib \
    h5py \
    pfapack \
    tqdm \
    pandas \
    polars \
    seaborn \
    fastparquet \
    pyarrow \
    opt-einsum \
    joblib \
    cloudpickle \
    networkx \
    sympy
```

Step 2 — Build and push:

```
docker build -t <dockerhub-username>/fermionic-env:latest .
docker push <dockerhub-username>/fermionic-env:latest
```

Step 3 — Convert to .sif on the access point:

```
# On ap20 (or ap21/ap23):
mkdir -p $HOME/tmp
export TMPDIR=$HOME/tmp
export APPTAINER_TMPDIR=$HOME/tmp
export APPTAINER_CACHEDIR=$HOME/tmp

apptainer build fermionic-env.sif \
    docker://<dockerhub-username>/fermionic-env:latest
```

For images > 1 GB, stage through OSDF rather than `transfer_input_files`. For quick iteration, reference the Docker Hub image directly in the submit file:

```
container_image = docker://<dockerhub-username>/fermionic-env:latest
```

5 Submitting 100 A100 Jobs

5.1 Submit File: `batch_gpu.submit`

```
universe = container
container_image = fermionic-env.sif

executable = run.sh
arguments = $(Process)

transfer_input_files = run.sh, fermionic-env.sif

log      = logs/job_$(Cluster)_$(Process).log
error    = logs/job_$(Cluster)_$(Process).err
output   = logs/job_$(Cluster)_$(Process).out

+JobDurationCategory = "Medium"

# Target A100 (Compute Capability 8.0, 40+ GB VRAM)
gpus_minimum_capability = 8.0
gpus_maximum_capability = 8.0
gpus_minimum_memory     = 40000

request_gpus    = 1
request_cpus    = 4
request_memory  = 16 GB
request_disk    = 10 GB

queue 100
```

Note: If you use a CVMFS path or a `docker://` URI for `container_image`, remove it from `transfer_input_files`. Only include local `.sif` files there.

5.2 Executable: `run.sh`

`$(Process)` runs 0–99. Use it to shard your parameter sweep or sample range:

```
#!/bin/bash
JOB_ID=$1

echo "Job ${JOB_ID} starting on $(hostname)"
echo "GPU: $(nvidia-smi --query-gpu=name --format=csv,noheader)"

python3 run_job.py --job-id ${JOB_ID}

chmod +x run.sh
mkdir -p logs
condor_submit batch_gpu.submit
```

5.3 Important Batch Submission Notes

- 100 jobs is well under the per-user 10,000-job queue limit.
- Jobs start as slots open across the pool, not simultaneously.
- Always use both \$(Cluster) and \$(Process) in log/output paths or files will overwrite each other.
- To cap how many run at once (e.g. to avoid storage contention): `max_idle = 20` in the submit file.

6 Mapping Your Workflows to OSPool Jobs

6.1 Pure-State Markov Snapshot Runs (colab_small_system_testing)

These notebooks call `classA_U1FGTN.gpu.run_markov_circuit()` for configurations like $N_x=16$, $N_y \in \{30, 40\}$, $n_{shell} \in \{1, 2\}$, 10 samples, 50 cycles. Each configuration is independent—natural to parallelize as separate HTCondor jobs.

A parameter sweep over (N_y , n_{shell}) might look like:

```
# generate_params.py -- run locally to create params.json
import json
params = []
for Ny in [30, 40]:
    for nshell in [1, 2]:
        params.append({"Ny": Ny, "nshell": nshell})
with open("params.json", "w") as f:
    json.dump(params, f)
```

```
#!/bin/bash
# run.sh -- receives job index as $1
JOB_ID=$1
python3 -c "
import json, sys
params = json.load(open('params.json'))
p = params[int('${JOB_ID}')]
print(p)
# call classA_U1FGTN_gpu with p['Ny'], p['nshell'], etc.
```

```
"
```

6.2 Large $N=20$ Entanglement Scaling (colab_large_entanglement_scaling_N20)

These notebooks run $N_x=20$, `samples=100`, $N_y \in \{30, 40, 50, 60, 80, 100, 120\}$, `nshell=1`, 50 cycles. Each N_y is independent. With 100 samples per N_y , you can shard samples across jobs:

```
# Submit 7 jobs, one per Ny value:
queue 7

# Or shard 100 samples as 10 jobs x 10 samples per Ny:
queue 70 # 7 Ny values * 10 jobs each
```

The GPU driver uses `complex128` by default. For $N_x=20$, $N_y=120$, the covariance matrix is $4N_xN_y/2 = 4800$ dimensional—comfortably within A100-40GB RAM.

6.3 Strip Entanglement Post-Processing

`strip_entropy_contours_gpu.py` and `strip_entropy_streaming_gpu.py` perform observable post-processing on saved covariance snapshots. These are also GPU-bound (matrix diagonalization via CUDA) and can be submitted as separate jobs after the snapshot runs complete, using DAGMan for dependency tracking:

```
# simple.dag
JOB   SNAPSOTS    snapshot_jobs.submit
JOB   STRIPS      strip_jobs.submit
PARENT SNAPSOTS CHILD STRIPS
```

```
condor_submit_dag simple.dag
```

7 Data Transfer

7.1 Small Files (<100 MB): transfer_input_files

```
transfer_input_files = run.sh, params.json, src/classA_U1FGTN_gpu.py
transfer_output_files = output_$(Process).npz
```

7.2 Larger Files: OSDF

For `.sif` containers, large `.npz` snapshots, or output data >100 MB, use the Open Science Data Federation (OSDF):

```
# Upload to OSDF staging area from access point:
osdf object put fermionic-env.sif \
    /osdf/users/asad.bhuiyan/fermionic-env.sif

# Reference in submit file:
container_image = osdf:///osdf/users/asad.bhuiyan/fermionic-env.sif
```

7.3 Transferring Results Back

```
# From your local machine or access point after jobs finish:
scp asad.bhuiyan@ap20.uw.osg-htc.org:~/results/ ./results/ -r
```

8 Common Pitfalls and Fixes

Problem	Fix
Job held immediately	Missing <code>+JobDurationCategory</code> . Add it.
Job runs on CPU node	Missing <code>request_gpus = 1</code> .
No A100s available	Remove <code>gpus_maximum_capability</code> to open up A10s too, or lower <code>gpus_minimum_capability</code> to 7.0 for V100s.
Output files overwrite each other	Use <code>\$(Cluster)_\$(Process)</code> in all output paths.
Job evicted after 20 hours	Implement HTCondor checkpointing or split into shorter segments.
<code>.sif</code> too large to transfer	Stage via OSDF instead of <code>transfer_input_files</code> .
<code>torch.cuda.is_available()</code> returns <code>False</code>	The container must be GPU-capable and <code>request_gpus ≥ 1</code> .
<code>pfpack</code> not found	Must build custom container; not in any pre-built OSPool image.

9 Full Working Example

This is a self-contained example for running 100 A100 jobs with your `fermionic_env` container.

`run.sh`:

```
#!/bin/bash
set -euo pipefail
JOB_ID=$1

echo "=== Job ${JOB_ID} on $(hostname) ==="
nvidia-smi --query-gpu=name,memory.total --format=csv,noheader

python3 - <<'EOF'
import sys, os, torch
import numpy as np

job_id = int(os.environ.get("JOB_ID", sys.argv[1] if len(sys.argv) > 1
    else "0"))
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Job {job_id} | device: {device} | GPU:
    {torch.cuda.get_device_name(0)}")

# --- your actual work here ---
# e.g.: import classA_U1FGTN_gpu; run_markov_circuit(...)
# save outputs to output_{job_id}.npz

np.savez_compressed(f"output_{job_id}.npz", job_id=job_id)
```

```
print(f"Job {job_id} done.")
EOF
```

batch_gpu.submit:

```
universe = container
container_image = docker://<dockerhub-username>/fermionic-env:latest

executable = run.sh
arguments   = $(Process)

environment = "JOB_ID=$(Process)"

transfer_input_files = run.sh

log      = logs/job_$(Cluster)_$(Process).log
error    = logs/job_$(Cluster)_$(Process).err
output   = logs/job_$(Cluster)_$(Process).out

+JobDurationCategory = "Medium"

gpus_minimum_capability = 8.0
gpus_maximum_capability = 8.0
gpus_minimum_memory     = 40000

request_gpus   = 1
request_cpus   = 4
request_memory = 16 GB
request_disk   = 10 GB

queue 100
```

Submit:

```
chmod +x run.sh
mkdir -p logs
condor_submit batch_gpu.submit
condor_watch_q
```

10 Quick Reference Card

```
# Login
ssh asad.bhuiyan@ap20.uw.osg-htc.org

# Submit
condor_submit job.submit

# Monitor
condor_q
condor_watch_q
condor_q -hold # why are jobs held?
```

```

# Cancel
condor_rm <cluster-id>
condor_rm $USER                # cancel everything

# GPU ClassAds (A100)
gpus_minimum_capability = 8.0
gpus_maximum_capability = 8.0
gpus_minimum_memory     = 40000

# Required tag
+JobDurationCategory = "Medium"    # Short / Medium / Long

# Pre-built GPU containers (CVMFS, no transfer needed)
# /cvmfs/singularity.opensciencegrid.org/htc/pytorch:2.3.1-cuda11.8
# /cvmfs/singularity.opensciencegrid.org/htc/rocky:9-cuda-12.6.0

# Build custom .sif
export TMPDIR=$HOME/tmp APPTAINER_TMPDIR=$HOME/tmp
    APPTAINER_CACHEDIR=$HOME/tmp
apptainer build myenv.sif docker://<user>/<image>:tag

# Help
support@osg-htc.org

```